Lecture 5.1: AWS Database

Maharishi International University

Department of Computer Science

M.S. Thao Huy Vu

Maharishi International University - Fairfield, Iowa

All rights reserved. No part of this slide presentation may be reproduced or utilized in any form or by any means, electronic or mechanical, including photocopying or recording, or by any information storage and retrieval system, without permission in writing from Maharishi International University (MIU).

Agenda

- **Introduction to Database Services in AWS**
- **Amazon RDS (Relational Database Service)**
- **Amazon Aurora**
- **Best Practices and Considerations**

Introduction to Database Services in AWS

- Services that provide database solutions tailored to different needs – from relational databases to NoSQL, in-memory, and graph databases
- Flexibility
- Durability and Availability
- Cost-efficiency
- Security

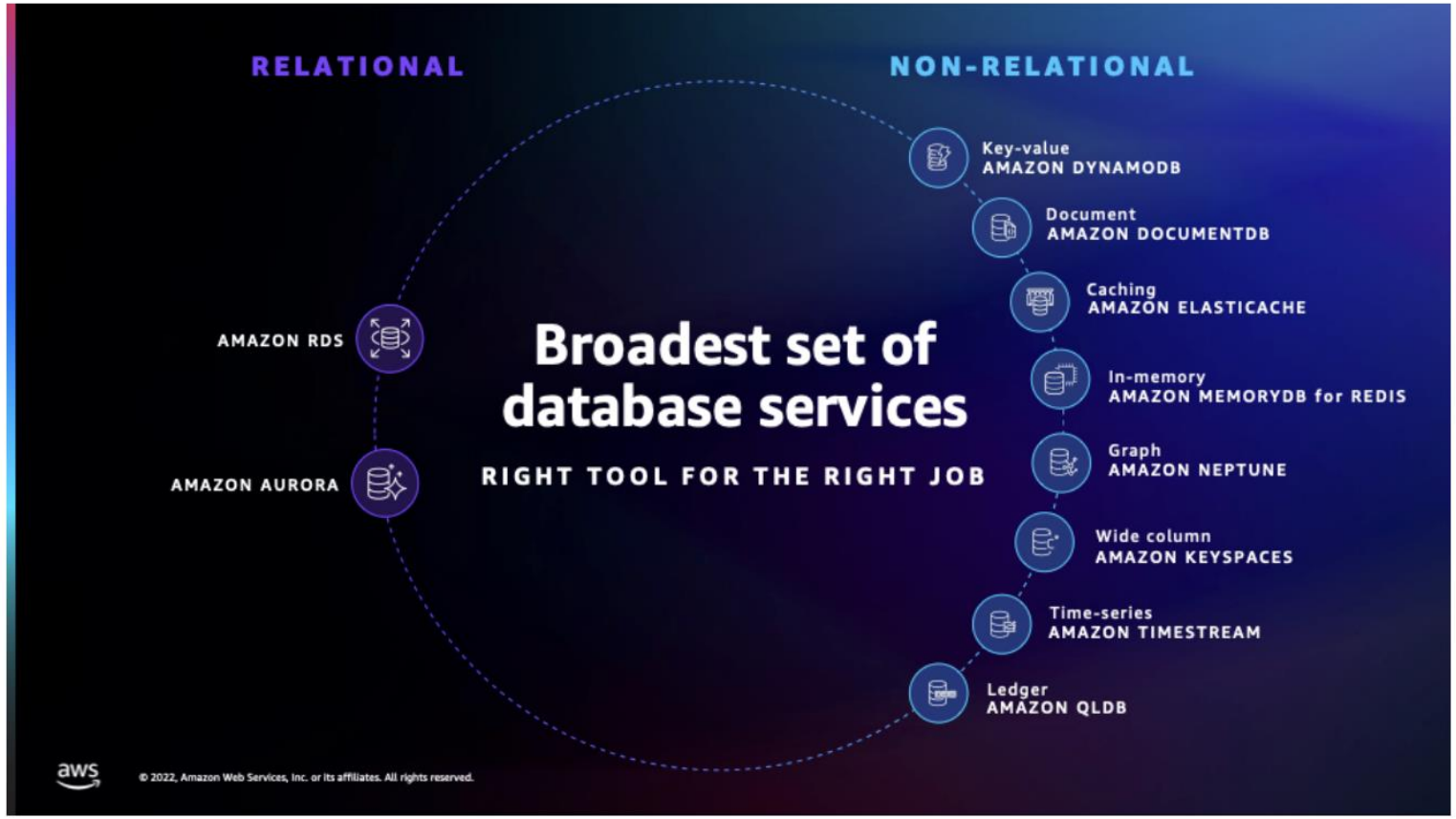
Introduction to Database Services in AWS

- Cloud Database vs. Self-hosted database servers
  - Backups
  - Patching
  - Scalability

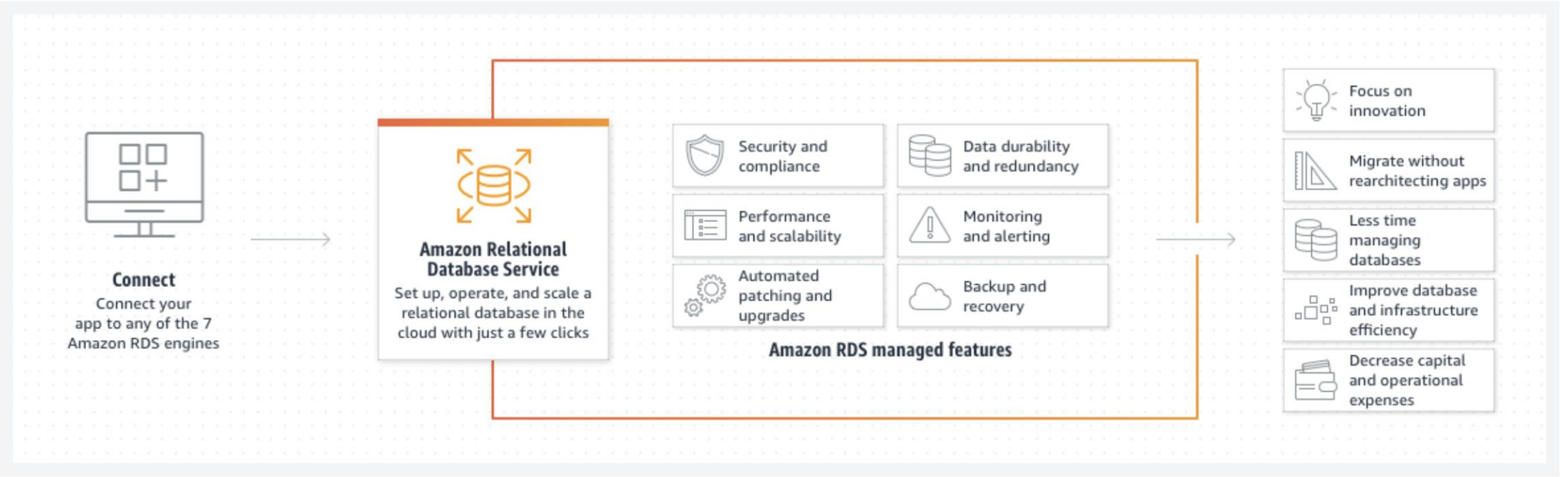
Introduction to Database Services in AWS

- Rational Databases: Amazon RDS, Amazon Aurora
- NoSQL Databases: DynamoDB, DocumentDB
- In-memory Databases: Amazon ElasticCache, Redis, Memcached
- Graph Databases: Amazon Neptune

Introduction to Database Services in AWS



Amazon RDS


Connect
Connect your app to any of the 7 Amazon RDS engines
Amazon Relational Database Service
Set up, operate, and scale a relational database in the cloud with just a few clicks
Security and compliance
Data durability and redundancy
Performance and scalability
Monitoring and alerting
Automated patching and upgrades
Backup and recovery
Amazon RDS managed features
Focus on innovation
Migrate without rearchitecting apps
Less time managing databases
Improve database and infrastructure efficiency
Decrease capital and operational expenses

Amazon RDS

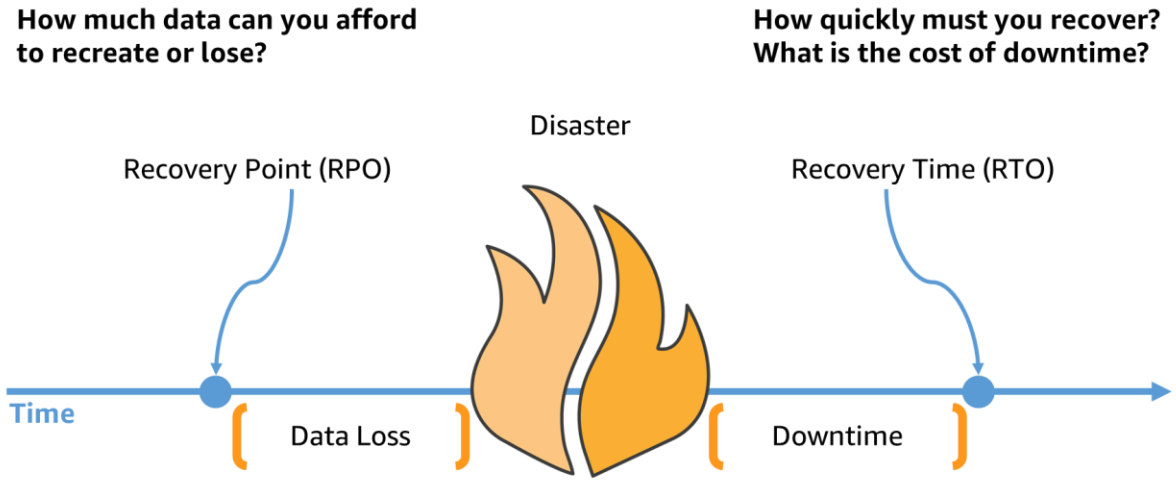
- RDS: Is a manage Relational Database Service that can host Relational Database
- Database engines: MySQL, PostgreSQL, Oracle, SQL Server, and MariaDB
- Multi-AZ deployment
- Read Replicas: For scaling out beyond the capacity constraints of a single DB instance for read-heavy database workloads
- Fault-tolerance: Automated Backups and Snapshots

Amazon RDS

- Security:
  - Encryption: Encryption at rest using AWS Key Management Service (KMS) or encryption in transit using SSL.
  - Network and Access Control: Using VPC for isolation or the security group of IAM.
- Performance and Monitoring:
  - Performance Insights
  - Scaling Options: Vertical vs. Horizontal or adding Read Replicas

RTO and RPO quick refresher

RTO (Recovery Time Objective) is a measure of how quickly after an outage an application must be available again. **RPO** (Recovery Point Objective) refers to how much data loss your application can tolerate. In RDS, RTO can be few minutes and RPO is quite low.



For more: [Establishing RPO and RTO Targets for Cloud Applications]

Multi-AZ deployment

Multi-AZ deployment increases availability by creating another DB instance in another AZ.

In RDS (Non-Aurora), it creates a **synchronous standby** instance or cluster.

1. Standby **instance** – You cannot read.
2. Standby **cluster** – You can read data. That means now you are not just paying double just in case the primary instance goes down.

In Aurora, it creates an **asynchronous read replica**.

During the outage, the standby instance (in non-Aurora) or read replica (in Aurora) is promoted to a primary instance in 2 minutes (RTO). During the automatic failover, the **URL stays the same**.

Standby Replica

In a Multi-AZ deployment, Amazon RDS (Non-Aurora) automatically provisions and maintains a **synchronous standby** replica in a different Availability Zone.

Standby replica provides

- data redundancy
- eliminate I/O freezes
- minimize latency spikes during system backups – The backup is taken from the standby instance.
- enhance availability during planned system maintenance


Region
Availability Zone
Amazon RDS
Amazon RDS standby replica
Availability Zone
Availability Zone
Amazon RDS
Amazon RDS DB instance

RDS Read Replicas

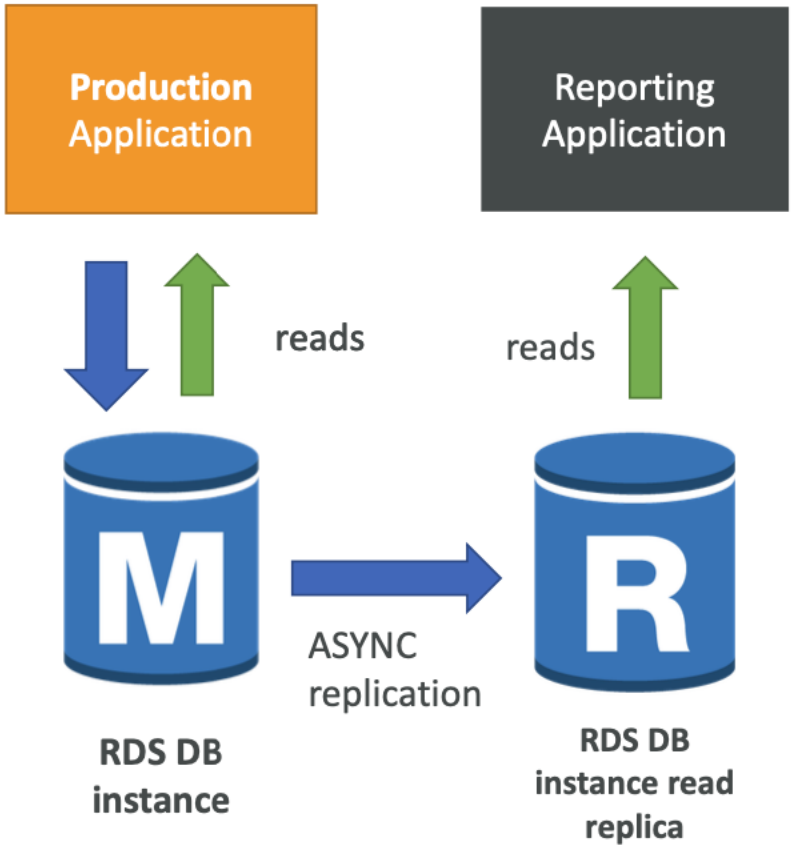
Amazon RDS Read Replicas improve

- enhanced performance
- scalability
- increased read throughput
- durability

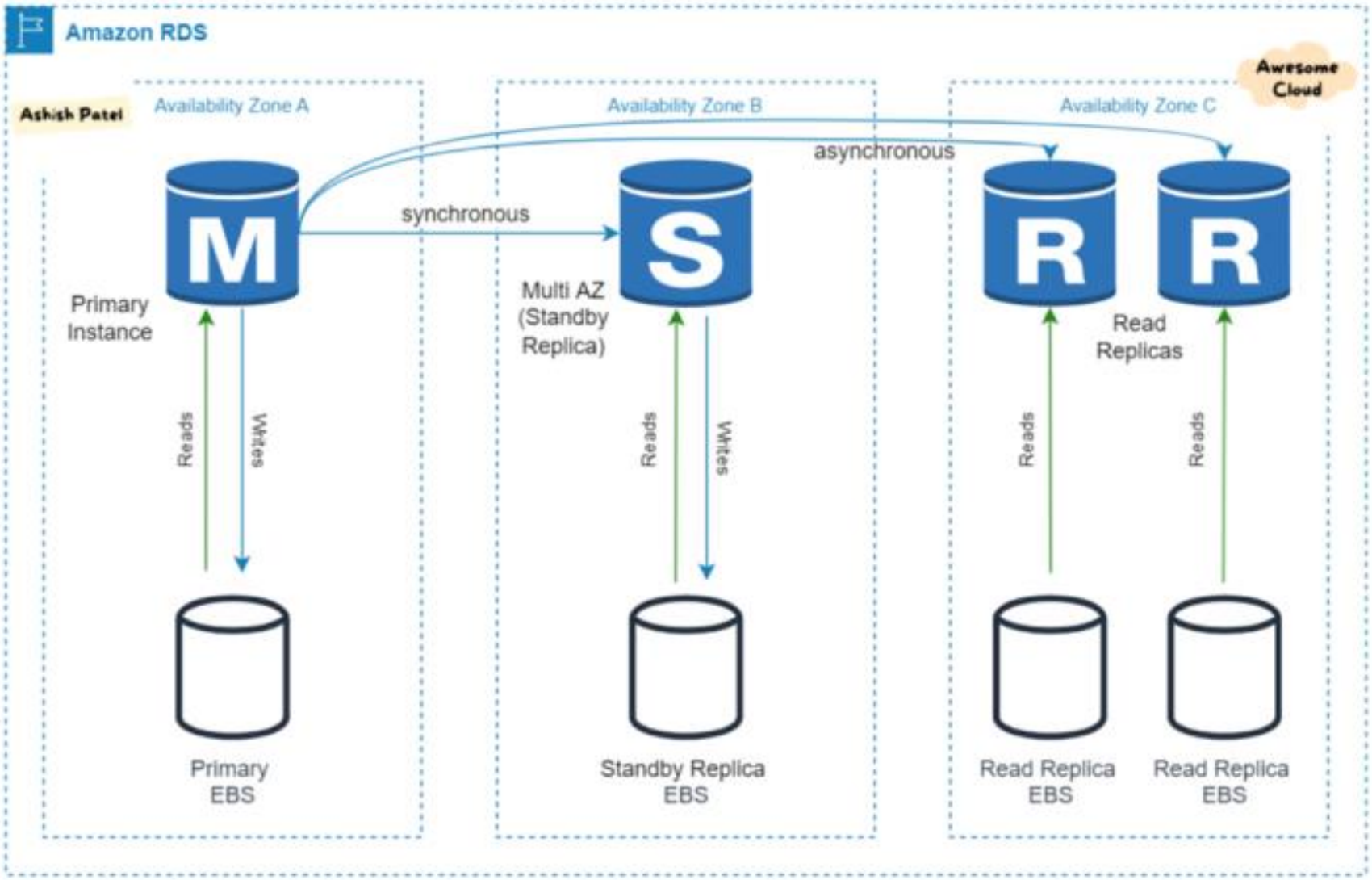
Replication is ASYNC, so reads are eventually consistent. All transactions are secure.

Replicas can be promoted to their own DB.

The Read Replicas be setup as **Multi AZ** for Disaster Recovery.

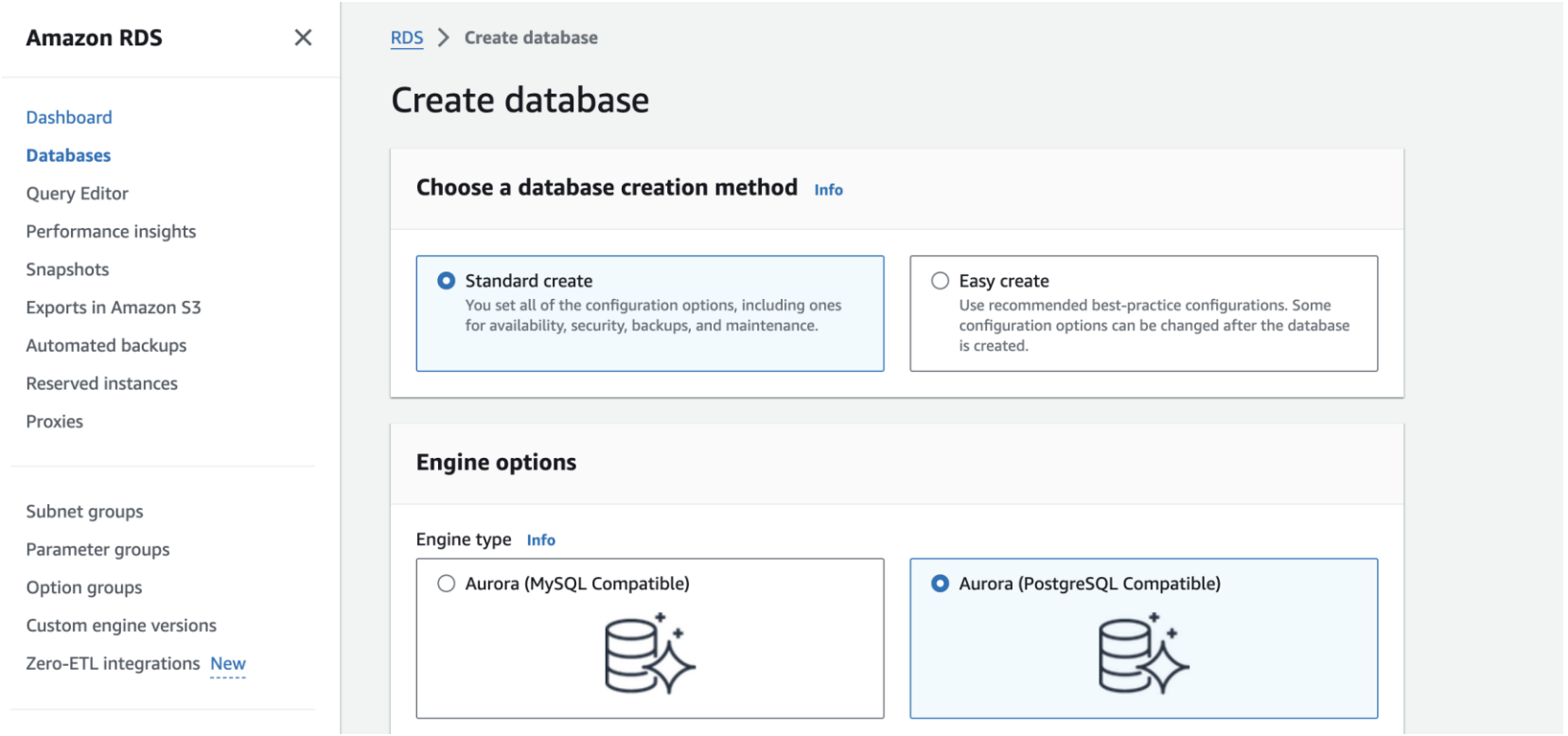


Read Replicas in Amazon RDS or Non-Aurora engines

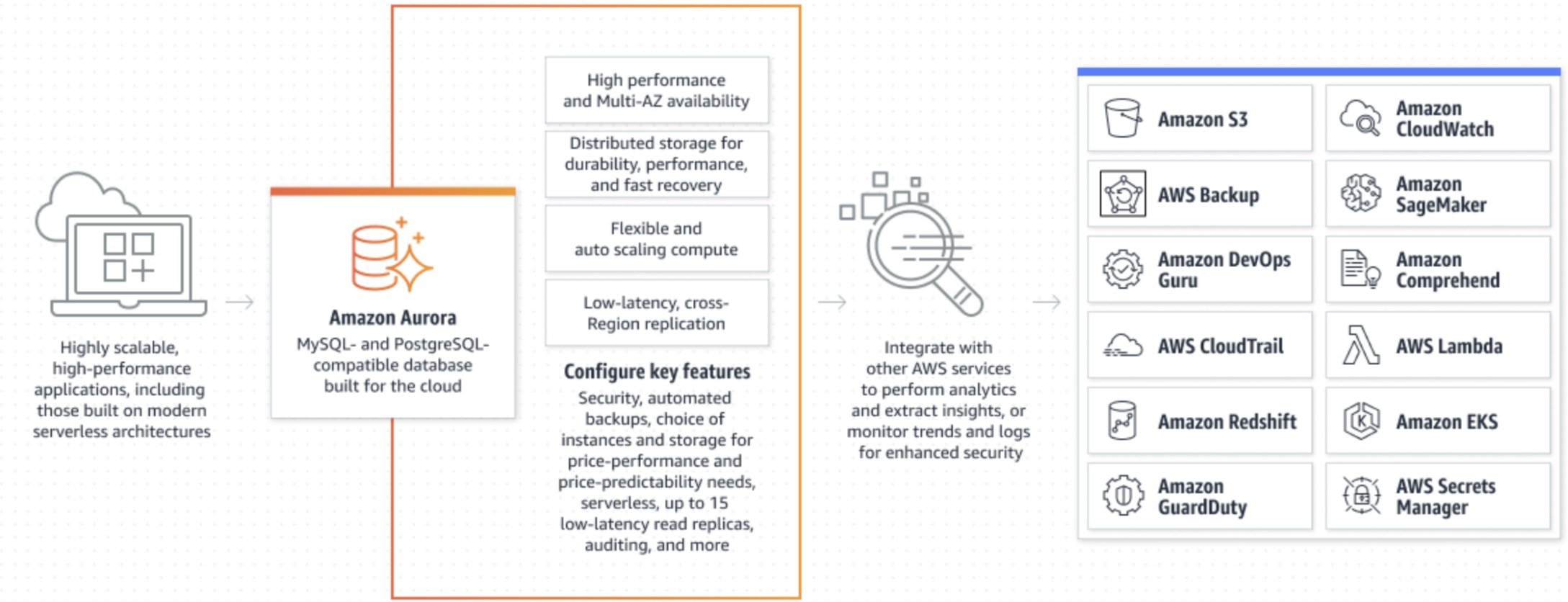


Amazon RDS

- Demo



Amazon Aurora



Amazon Aurora

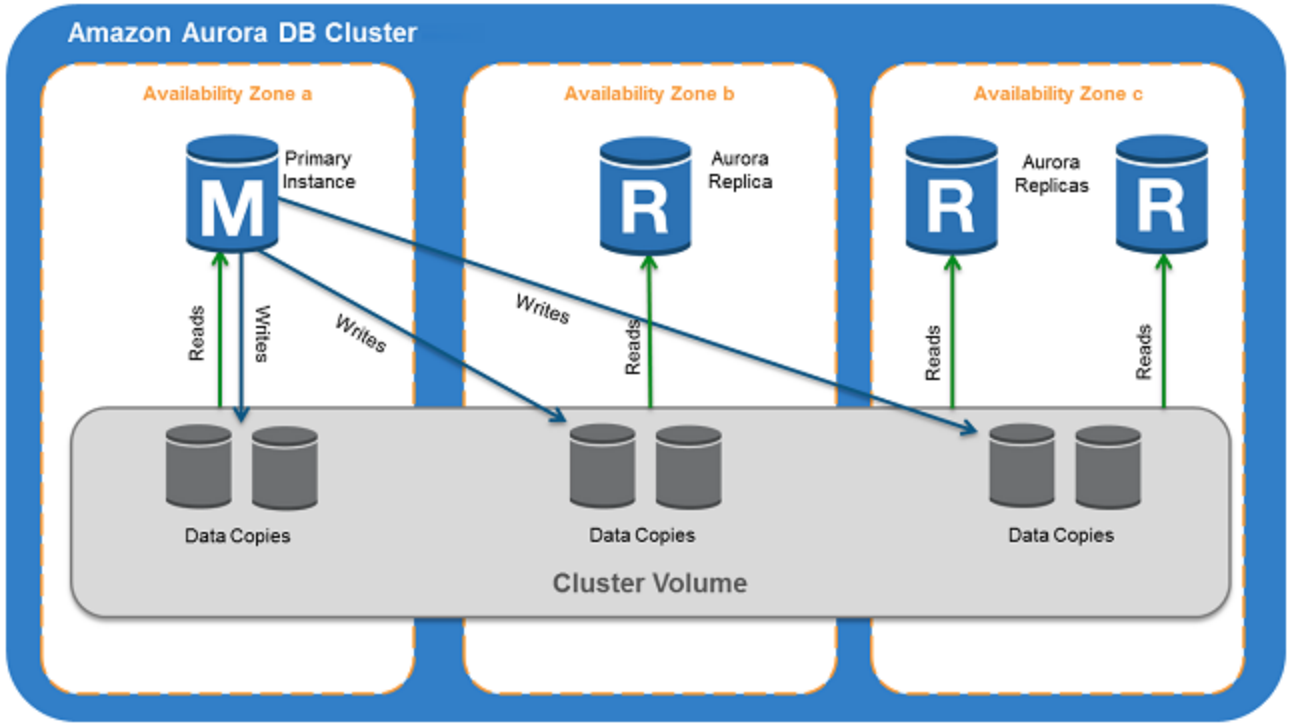
- A service for **MySQL** and **PostgreSQL** compatible relational database built for the cloud
- Scalability
- High performance
- Durability & Availability
- Security

Amazon Aurora

- Distributed, scale-out architecture
- Storage Subsystem: Automatically replicate across three AZs
- Endpoints: Different types of endpoints such as cluster endpoint, reader endpoint, and custom endpoint for load-balancing purpose

Amazon Aurora

- It contains a primary instance and multiple replicas



Amazon Aurora DB clusters

An Amazon Aurora DB cluster consists of one or more **DB instances** and a cluster **volume.** An Aurora **cluster volume** is a virtual database storage volume that spans **3 AZs**, with each AZ having a copy (**replication**) of the DB cluster data. Two types of DB instances make up an Aurora DB cluster:

1. **Primary DB instance** – Supports read and write operations, and performs all of the data modifications to the cluster volume. Each Aurora DB cluster has one primary DB instance.
2. **Aurora Replica** – Connects to the same storage volume as the primary DB instance and supports **only read** operations. Replication is done in **10** ms.

Amazon Aurora Read Replica

Amazon Aurora replicas **share the same underlying storage** as the source instance, lowering costs and **avoiding the need to copy data** to the replica nodes.

Each Aurora DB cluster can have up to 15 Aurora Replicas in addition to the primary DB instance. Maintain **high availability** by locating Aurora Replicas in separate Availability Zones.

Aurora automatically **fails over** to an Aurora Replica in case the primary DB instance becomes unavailable in 30 seconds.

Amazon Aurora Read Replica and Fail-Over

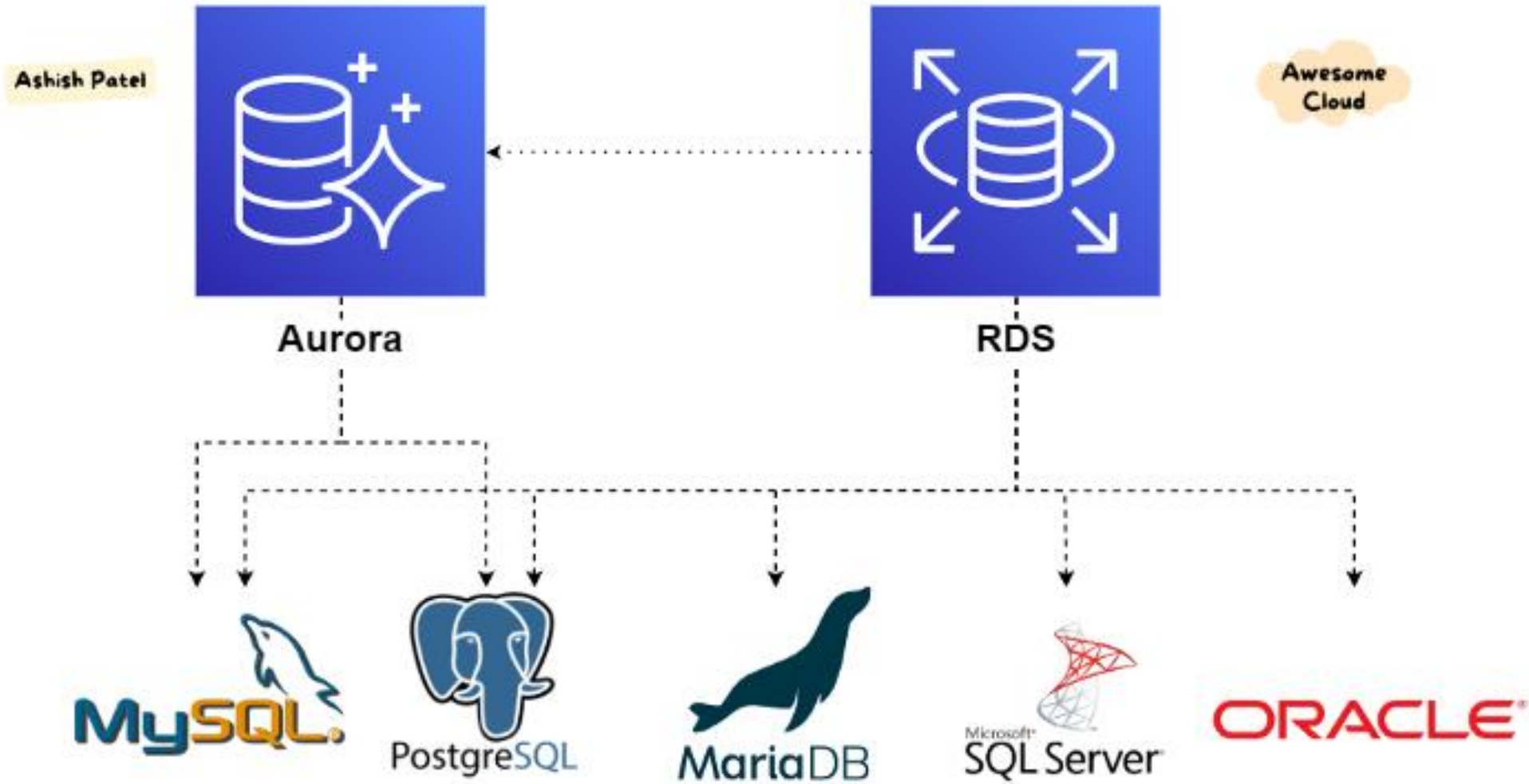
Amazon Aurora doesn’t copy or transfer any data during the replication. Instead, just share the underlying storage which helps reach the **minimal replica lag**. This lag is usually much less than 10 milliseconds after the primary instance has written an update.

If the primary instance in a DB cluster using single-master replication fails, Aurora automatically fails over to a new primary instance in one of two ways:

1. By promoting an existing Aurora Replica to the new primary instance (in 30 seconds)
2. By recreating a new primary instance if there isn’t any replicas (10 minutes)

RDS vs Aurora

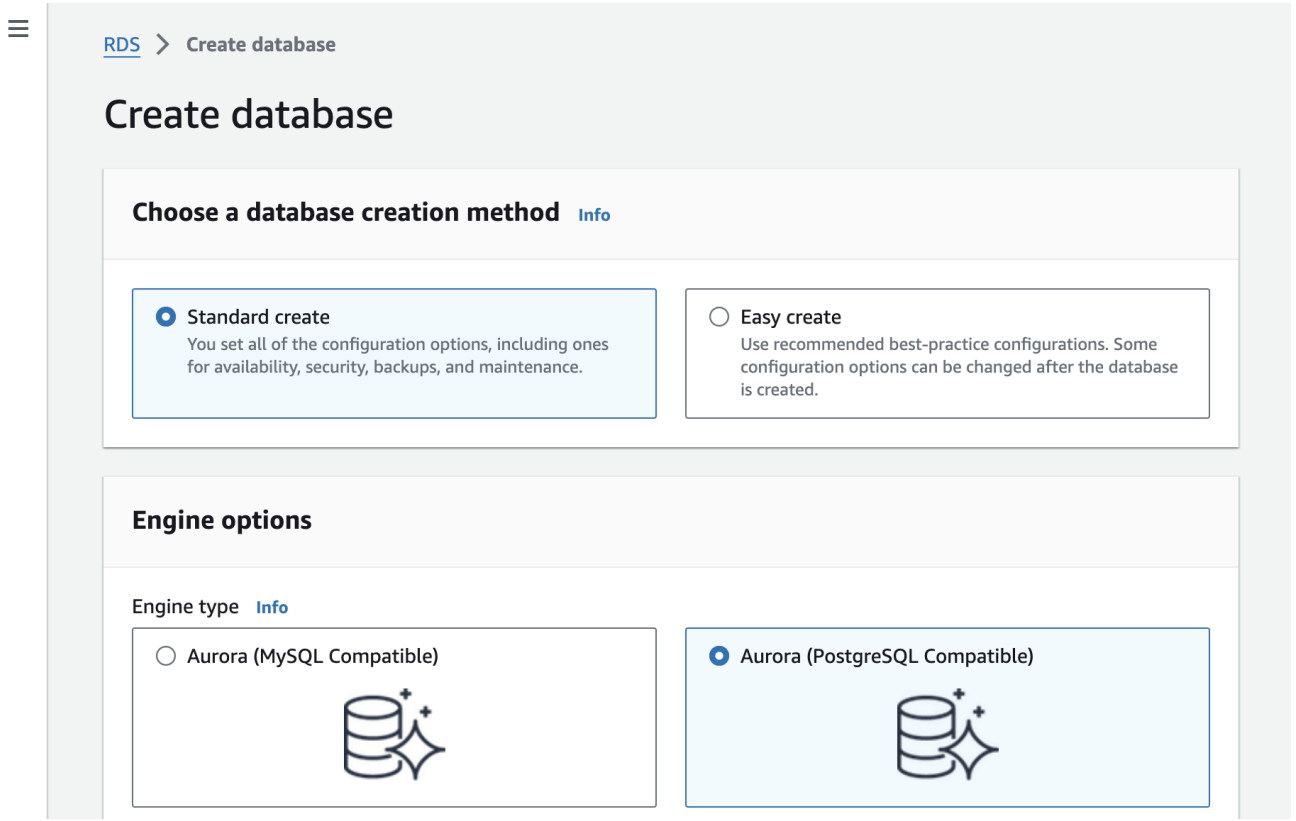
	Multi-AZ deployment (Fail over)	Scalability	Storage Volume
RDS	Standby instance (sync)	Read replica	Inside the instance
Aurora	Read replica (async)	Read replica (async)	Shared (one cluster volume)



Source: AWS — Difference between Amazon Aurora and Amazon RDS

Amazon Aurora

- Demo



Best practices and considerations

- Security:
  - Encryption: Encrypt data both at rest using AWS Key Management Service (KMS) and in transit using SSL/TLS to protect sensitive data.
  - Access Control: Implement least privilege access using IAM to control who can manage and access database resources.
  - Network Security: Advocate for hosting databases in private subnet within a VPC to isolate them from the public internet and utilize security groups and network ACLs effectively.

Best practices and considerations

- Performance Optimization
  - Right sizing: choose the correct instance size and database type based on the workload's specific needs for appropriate provisioning.
  - Indexing and queries: Using indexing strategies and writing efficient SQL queries to improve performance.
  - Monitoring and tuning: Regularly monitor databases to identify the performance bottlenecks and tune the database accordingly by using tools like CloudWatch or Performance Insights.

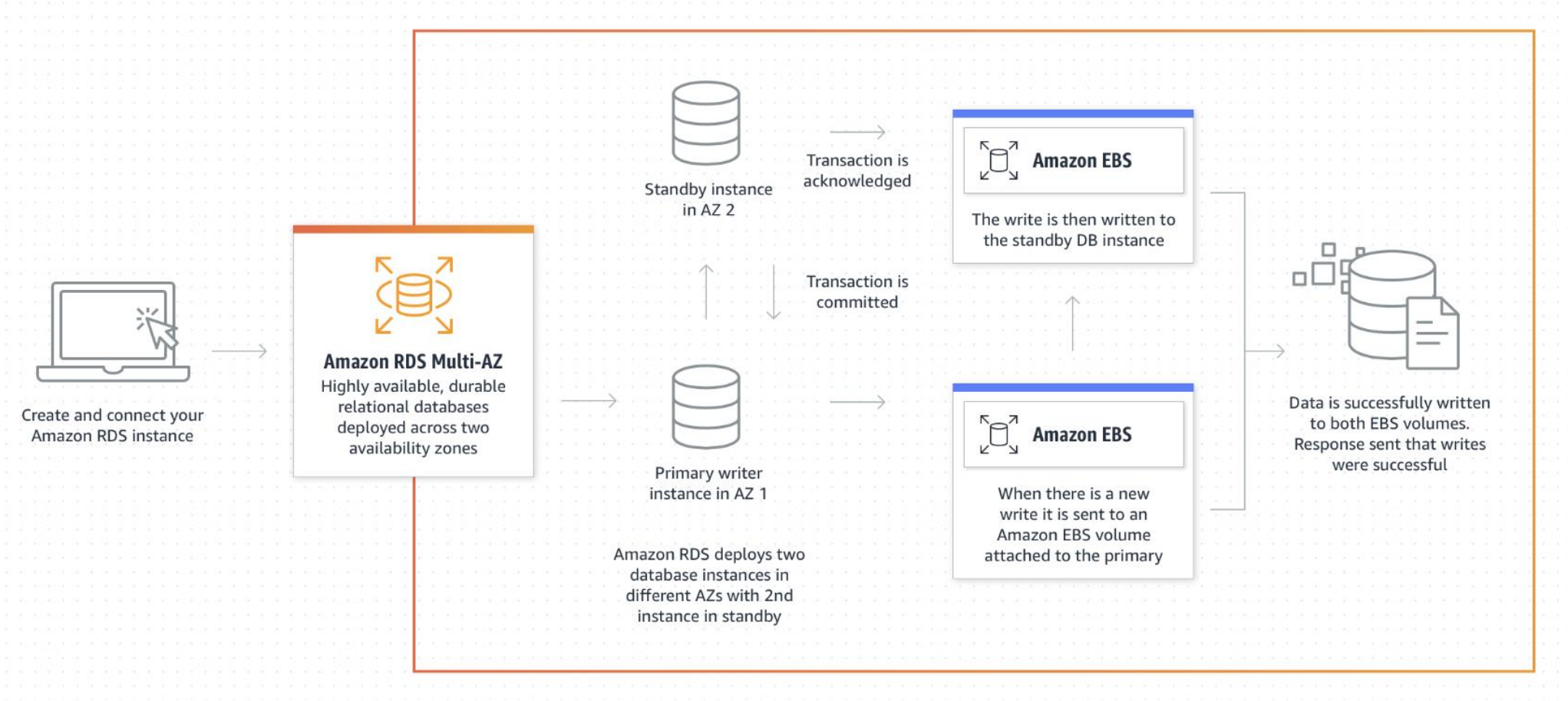
Best practices and considerations

- High Availability and Disaster Recovery
  - Multi-AZ deployments: Deploy databases across multiple AZs to ensure high availability and failover protection.
  - Backup and Restore: Regular backups, point-in-time recovery, and testing the restore process to ensure data durability and recoverability.
  - Read Replicas: For read-heavy database workloads, recommend the use of read replicas to enhance application performance and distribute loads.

Best practices and considerations

- Cost Management:
  - Managed Services: Select the right types of services like Amazon RDS or Aurora
  - Storage Management: Select the appropriate storage type and use automated storage scaling
  - Reserved Instances: Purchasing Reserved Instances for databases with predictable workloads can save up to 75% over equivalent on-demand capacity.

Best practices and considerations



References

- https://docs.aws.amazon.com/
- ChatGPT: https://chatgpt.com/
- Google AI: https://gemini.google.com/app